

Estructuras de Datos - Ejercicio Tipo Examen

Ejercicio Tipo Examen

Enunciado: Dado el grafo de comunicaciones y el grafo de transferencias representados en la red de corrupcion investigada por la fiscalia:

- Implementa el metodo `getContactos` para obtener la lista de contactos de una persona especifica, ordenados de acuerdo con la frecuencia de sus comunicaciones (de mayor a menor), junto con la cantidad de dinero enviado y recibido.
- Implementa el metodo `detectarInteraccionesSospechosas` para identificar pares de individuos que se han comunicado al menos un numero n de veces y que han transferido dinero entre si a traves de intermediarios.

Resolucion del Metodo `getContactos`

Analisis: El metodo `getContactos` se basa en:

- Recorrer las listas de adyacencia del grafo de comunicaciones para contar las interacciones.
- Utilizar el grafo de transferencias para sumar los valores enviados y recibidos entre el individuo en cuestion y sus contactos.
- Ordenar los contactos por la cantidad de interacciones y devolver la lista ordenada con los valores calculados.

Codigo en Java:

```
1 public class Grafo {
2     private Map<String, List<String>> comunicaciones;
3     private Map<String, Map<String, Double>> transferencias;
4
5     public Grafo() {
6         comunicaciones = new HashMap<>();
7         transferencias = new HashMap<>();
8     }
9
10    public void agregarComunicacion(String origen, String destino) {
11        comunicaciones.putIfAbsent(origen, new ArrayList<>());
12        comunicaciones.get(origen).add(destino);
13    }
14
15    public void agregarTransferencia(String origen, String destino, double monto) {
```

```

16     transferencias.putIfAbsent(origen, new HashMap<>());
17     transferencias.get(origen).put(destino, transferencias.get(origen).
        getOrDefault(destino, 0.0) + monto);
18 }
19
20 public List<String> getContactos(String nombre) {
21     Map<String, Integer> frecuencia = new HashMap<>();
22     Map<String, Double> recibido = new HashMap<>();
23     Map<String, Double> enviado = new HashMap<>();
24
25     // Contar comunicaciones
26     for (String destino : comunicaciones.getOrDefault(nombre, new ArrayList<>())
        ) {
27         frecuencia.put(destino, frecuencia.getOrDefault(destino, 0) + 1);
28     }
29
30     // Procesar transferencias
31     for (Map.Entry<String, Map<String, Double>> entry : transferencias.entrySet
        ()) {
32         String origen = entry.getKey();
33         for (Map.Entry<String, Double> destino : entry.getValue().entrySet()) {
34             if (destino.getKey().equals(nombre)) {
35                 recibido.put(origen, recibido.getOrDefault(origen, 0.0) +
                    destino.getValue());
36             } else if (origen.equals(nombre)) {
37                 enviado.put(destino.getKey(), enviado.getOrDefault(destino.
                    getKey(), 0.0) + destino.getValue());
38             }
39         }
40     }
41
42     // Generar y ordenar la lista de contactos
43     List<String> contactos = new ArrayList<>(frecuencia.keySet());
44     contactos.sort((a, b) -> frecuencia.get(b) - frecuencia.get(a));
45
46     // Imprimir resultados
47     System.out.println("Contacto \t Recibido \t Enviado");
48     for (String contacto : contactos) {
49         System.out.printf("%s \t %.2f \t %.2f\n", contacto,
50             recibido.getOrDefault(contacto, 0.0),
51             enviado.getOrDefault(contacto, 0.0));
52     }
53     return contactos;
54 }
55 }
56
57 public static void main(String[] args) {
58     Grafo grafo = new Grafo();
59
60     // Agregar comunicaciones
61     grafo.agregarComunicacion("Andersley B.", "Monara V.");
62     grafo.agregarComunicacion("Andersley B.", "Rafian C.");
63
64     // Agregar transferencias
65     grafo.agregarTransferencia("Andersley B.", "Monara V.", 15000);
66     grafo.agregarTransferencia("Monara V.", "Andersley B.", 7600);
67
68     // Obtener contactos

```

```
69     grafo.getContactos("Andersley B.");
70 }
```

Resolucion del Metodo detectarInteraccionesSospechosas

Analisis: Este metodo identifica pares de individuos que cumplen:

- Se han comunicado al menos n veces.
- Existe una transferencia de dinero entre ellos a traves de uno o mas intermediarios.

Codigo en Java:

```
1 public List<String> detectarInteraccionesSospechosas(int n) {
2     List<String> sospechosos = new ArrayList<>();
3
4     for (String origen : comunicaciones.keySet()) {
5         for (String destino : comunicaciones.get(origen)) {
6             if (contarComunicaciones(origen, destino) >= n && hayTransferencia(
7                 origen, destino)) {
8                 sospechosos.add(origen + " - " + destino);
9             }
10        }
11    }
12
13    return sospechosos;
14 }
15
16 private int contarComunicaciones(String origen, String destino) {
17     return (int) comunicaciones.getDefault(origen, new ArrayList<>()).stream()
18         .filter(dest -> dest.equals(destino)).count();
19 }
20
21 private boolean hayTransferencia(String origen, String destino) {
22     // Implementar busqueda de camino con transferencias intermedias
23     return false; // Placeholder para busqueda de rutas con pesos
24 }
```